SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**
Question Count: 5

Code of Conduct

I K. Krishna Chaitanya (192372241) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Krishna Chaitanya (192372241)

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and	Most constraints identified	Some constraints identified	Constraints poorly understood

		prioritized accurately			
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

SOLUTIONS

Task 1: HDFS Storage Solution for Media Platform

Technical Architecture

- **Storage Tiering (Heterogeneous Storage):** Implement HDFS storage policies combining HOT (NVMe/SSD for daily active streams), WARM (standard HDDs for catalog media), and COLD/ARCHIVE tiers (high-capacity dense disks for legacy media).
- **Hadoop 3.x Erasure Coding (EC):** Apply RS-6-3 (Reed-Solomon) Erasure Coding on warm and cold media tiers. This reduces storage overhead from 300% (traditional 3 × replication) down to 150% (1.5 ×), cutting disk costs by 50%.
- **Rack Awareness & HA:** Configure multi-rack deployment with HDFS NameNode High Availability via Quorum Journal Manager (QJM) to eliminate single points of failure.

Constraint Satisfaction & Justification

- **Availability > 99.9%:** Hot ingest uses standard replication factor of 3 with rack-aware placement (1 replica on local rack, 2 on remote rack) ensuring zero downtime during single-node or rack failures. NameNode automatic failover via ZooKeeper ensures continuous metadata operations.
- **Replication Factor ≤ 3:** The hot layer strictly uses 3 × replication, while older media transitions via lifecycle policies to Erasure Coding, maintaining equivalent fault tolerance (3 parity blocks) without exceeding storage limits.
- **Cost-Efficient Growth:** Erasure Coding combined with storage tiering minimizes raw drive requirements as media volume scales exponentially.

Task 2: MapReduce Job Optimization for Strict 20-Minute SLA

Job Design & Task Distribution

- **Input Splitting & Sizing:** Configure mapreduce.input.fileinputformat.split.maxsize to match optimal block sizes (128MB/256MB) and implement CombineFileInputFormat to merge small log files, preventing mapper initialization overhead.
- **In-Memory Map-Side Operations:**
  - Implement an in-mapper **Combiner** (Reducer logic executed locally) to aggregate repetitive log events (e.g., status codes, user hits) before disk spill.
  - Adjust mapreduce.map.sort.spill.percent = 0.85 and mapreduce.task.io.sort.mb = 512MB to reduce I/O bottlenecks and intermediate disk spills.
- **Custom Partitioning & Speculative Execution:**

    - Apply a custom HashPartitioner on hashed session IDs to balance key distribution and eliminate Reducer stragglers.
    - Enable mapreduce.map.speculative = true and mapreduce.reduce.speculative = true to re-launch lagging tasks on alternate nodes.
- **Intermediate Compression:** Enable Snappy compression (mapreduce.map.output.compress = true with SnappyCodec) to cut network serialization and shuffle transfer latency by up to 60%.

SLA Justification

- Minimizing shuffle data via combiners and Snappy compression allows the job to complete within the 20-minute window despite limited physical cluster bandwidth.

Task 3: Multi-Workload YARN Scheduler Configuration

Scheduler Selection & Queue Hierarchy

Implement the **YARN Capacity Scheduler** configured with hierarchical queues and preemption enabled (yarn.resourcemanager.scheduler.monitor.enable = true).

Queue Name	Min Capacity (%)	Max Capacity (%)	Priority Level	Preemption Policy
root.etl	50%	100%	High (Level 1)	Cannot be preempted; preempts others
root.ml	30%	60%	Medium (Level 2)	Can be preempted down to min capacity
root.reporting	20%	40%	Low (Level 3)	Can be preempted

Constraint Handling

- **ETL Priority:** If the cluster is saturated and an ETL batch arrives, the Capacity Scheduler preempts resources from reporting and ml using yarn.resourcemanager.monitor.capacity.preemption.total_preemption_per_round = 0.1.
- **Fairness:** When ETL is idle, ML and reporting workloads can burst up to their maximum capacity limits. Minimum guaranteed capacities prevent resource starvation across departments.
- **Node Failure Recovery:** Set ApplicationMaster retries (yarn.resourcemanager.am.max-attempts = 3) and enable Work-Preserving ResourceManager Restart (yarn.resourcemanager.work-preserving-recovery.enabled = true). If a worker node crashes, containers are reallocated automatically on healthy nodes without restarting entire pipelines.

Task 4: Unified Ingestion Architecture (SAN, NAS, RDBMS)

Component Selection & Constraints

- **RDBMS Ingestion (Apache Sqoop / CDC):** Incremental imports using append/last-modified modes capture transactional deltas without full-table re-ingestion, keeping load minimal.
- **NAS Ingestion (Apache Flume / NiFi):** Spooling directory listeners stream file shares continuously to HDFS, avoiding manual bulk copy bottlenecks.
- **SAN Ingestion (Direct Fiber Channel Gateway):** Mount-to-HDFS parallel streaming jobs convert block storage archives into sequenced Avro/Parquet objects.
- **Deduplication Strategy:** Ingested streams pass through a lightweight staging layer using Apache Spark/Hive with bloom filters and key-hashing (SHA-256) to drop duplicate records before writing to raw storage.
- **Backup Reliability:** Implement **HDFS Snapshotting** (hdfs dfsadmin -allowSnapshot) on target directories for zero-overhead point-in-time recovery, paired with asynchronous offsite replication using DistCp.

Task 5: Secure, High-Availability End-to-End Hadoop Solution

Design Implementation

- **Secure Data Movement:**

    - **Perimeter Defense:** Apache Knox acts as the central reverse proxy providing perimeter security, SSL termination, and LDAP/SSO integration.
    - **Authentication & Authorization:** MIT Kerberos provides end-to-end mutual authentication (RPC encryption enabled). Apache Ranger manages centralized role-based access control (RBAC) and column-level masking.
    - **Data at Rest:** HDFS Transparent Data Encryption (TDE) integrated with Hadoop Key Management Server (KMS).
- **High Availability (HA):**
    - Active/Standby NameNodes synchronized via 3-node Quorum Journal Manager (QJM) and automated ZooKeeper Failover Controllers (ZKFC).
    - Dual YARN ResourceManagers configured with active/standby state store in ZooKeeper.
- **NiFi / ETL Integration:**
    - Secured Apache NiFi Site-to-Site (S2S) protocol moves data across networks via mutual TLS.
    - Native NiFi PutHDFS processors authenticate directly via Kerberos keytabs to stream structured and semi-structured payloads securely into encrypted HDFS zones.